



An efficient Ant Colony algorithm based on wake-vortex modeling method for aircraft scheduling problem



Bo Xu

Business School, University of Shanghai for Science and Technology, Shanghai 200093, China

ARTICLE INFO

Article history:

Received 17 August 2016

Received in revised form 21 November 2016

Keywords:

Aircraft scheduling problem

WVM

Rank 2 matrix approximation

Mixed-integer programming

ABSTRACT

The aircraft scheduling problem (ASP) is a salient problem in airport runway scheduling system. This paper originally proposes an Ant Colony (AC) algorithm based on the wake-vortex modeling (WVM) method for ASP. Numerical results validate that this new method has better performance than CPLEX, general AC algorithm, and approximation algorithm in Ma et al. (2014). It is a promising method to improve the efficiency of the aircraft scheduling system from a theoretical standpoint.

© 2016 Elsevier B.V. All rights reserved.

1. Introduction

With the rapidly increasing demand, air traffic is expected to double in the world over the next 15 years, even triple in some regions. As a result, many big airports are facing or will face the significant risk of air traffic delays [1]. How to enhance the airports' capacity and throughput has become a severe problem (Boehme [2], Arkind [3]).

In the current airport system, runway is identified as one of the primary bottleneck (Idris et al. [4]). That is to say, even small enhancements to the runway throughput will significantly improve the airports' efficiency. From this point, many scholars pay their attentions on the runway system. One of the most important operational problems is the aircraft scheduling problem (ASP).

ASP aims to make the most use of the runway by scheduling the take-off/landing of aircraft, i.e. to take off/land as many aircraft as possible within a given period of time. ASP concerns a given minimum separation time (MST) table. Taking the aircraft landing problem for example, Table 1 illustrates the MST between the successive aircraft concerning three typical aircraft types (small, large and heavy). If a small aircraft lands after a heavy one, the required separation between them is 196 s; however, when a heavy aircraft lands after a small, the separation is only 60 s. So a proper scheduling strategy may shorten the makespan. Fig. 1 presents two scheduling strategies to the same three aircraft. Obviously, the strategy of sequences 2 saves more than 50% of time for runway than that of sequence 1.

The ASP is a NP-hard problem since there are at most $A_n^n = [n(n-1)(n-2) \cdots 1]$ possible sequencing strategies with n aircraft (Beasley et al. [5], Bianco et al. [6], Garey and Johnson [7]). To cope with it, branch and bound (B&B), ant colony (AC), dynamic programming (DP), approximation method and many other methods are often adopted by researchers.

Branch and Bound: Brinton [9] has introduced a B&B approach for aircraft landing problem in 1992. Then Abela et al. [10], Ernst et al. [11] and Beasley et al. [5] enhance the efficiency of B&B and extend it to multi-runway case. Bennell et al. [12] present an extensive literature overview on the B&B. Recently, Faye [13] presents a B&B algorithm based on rank 2 matrix approximation method for the multi-runway scheduling problem.

E-mail address: 1986xubo@gmail.com.

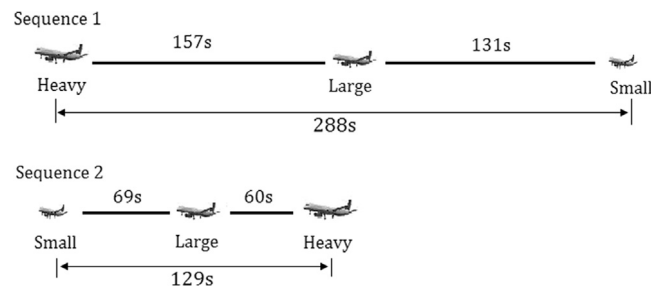


Fig. 1. Two ordering strategies result in different makespan for the same three aircraft.

Table 1

MST (in sec.) between successive landing aircraft on the same runway for three aircraft types (Lee [8]).

		Following aircraft types		
		1	2	3
Leading aircraft types	1	82	69	60
	2	131	69	60
	3	196	157	96

1-small aircraft, 2-large aircraft, 3-heavy aircraft.

Ant colony: Ant colony (AC) is firstly used by Randall [14] for runway scheduling in 2002. The objective function is to minimize the total penalty costs. Then Bencheikh et al. [15,16] improve AC by incorporating a local search heuristic and extend AC to tackle the multi-runway case. Zhan et al. [17] design an AC system in real-time scenario for ASP.

Dynamic programming: The dynamic programming (DP) for runway scheduling is firstly developed by Dear [18] in constraint position shift (CPS) model. Then several researchers have used DP and CPS to model fairness, to develop fast solution techniques, and to incorporate many operational constraints (Psaraftis [19], Dear and Sherif [20], Neuman and Erzberger [21], Trivizas [22], de Neufville and Odoni [23], Carr [24], Balakrishnan and Chandran [25]). Recently, Lieder et al. [26] present a DP algorithm via making use of the aircraft classes.

Approximation method: The approximation method is proposed by Ma et al. [27] and Faye [13]. Ma et al. [27] use a rank 2 matrix to approximate the original separation matrix. In this manner, the ASP becomes sequence-independent and thus easier to solve. Faye [13] combines the rank 2 matrix approximation method with the B&B algorithm for the multi-runway scheduling problem.

Other methods: Some other methods for runway and aircraft scheduling can be seen in Anagnostakis et al. [8], Atkins and Brinton [28], Bohme [29], Lee [30], Willemain et al. [31], De Maere and Atkin [32], Ji et al. [33] and Lieder and Stollatz [34].

In this paper, we go on investigating the matrix approximation method, and combining it with the AC algorithm to solve ASP. AC is chosen here since each ant in AC directly returns a sequencing strategy which is a possible choice to increase the precision of the approximation method. We firstly generate a rank 2 matrix (denoted by MST') by the Wake-vortex modeling (WVM) method. Then MST' is approximating to the actual minimum separation time (MST), where the optimal solution concerning MST' is easy to find when the aircraft ready time constraint is ignored. Finally we use AC to optimize the ASP while considering the differences between MST and MST' and adding the aircraft ready time constraint. The numerical study validates that this new method has better performance than CPLEX, general AC algorithm, and approximation algorithm in Ma et al. [27]. It is a promising method to improve the efficiency of the aircraft scheduling system from a theoretical standpoint.

In the remainder of this paper, we follow the classic *triangle inequality* assumption of MST, i.e. $d_{ij} \leq d_{ik} + d_{kj}$, $\forall i, j, k \in \Lambda$, where Λ is the aircraft type set, and d_{ij} is the MST of aircraft with type i followed by that with type j . We use **WVM-AC** to denote “the AC algorithm based on WVM method” for abbreviation in this paper.

This paper is organized as follows. The ASP is defined in Section 2. In Section 3, the MST constraint for ASP is analyzed, and the WVM method is proposed. In Section 4, WVM-AC algorithm is originally designed to solve ASP. The numerical study is conducted in Section 5, while some conclusions are summarized in Section 6.

2. Problem definition

Suppose there are n aircraft, and they are labeled according to their order in the First Come First Serve (FCFS) sequence. Θ denotes the aircraft set. We consider the case that the runway is only used for the aircraft landing. π is the final landing sequence which indicates the order of all the aircraft. π_i , the i th element in sequence π , denotes the order of aircraft i in the original (FCFS) sequence. For example, $\pi_6 = 5$ means that the sixth aircraft in the scheduled sequence π is the fifth aircraft in the FCFS sequence before scheduling. $TP(\pi_i)$ is the type of aircraft π_i , and $d_{TP(\pi_i)TP(\pi_{i+1})}$ denotes the MST between two successive aircraft (π_i and π_{i+1}) in sequence π . e_{π_i} is the earliest landing time (ready time) of aircraft π_i . Then the landing

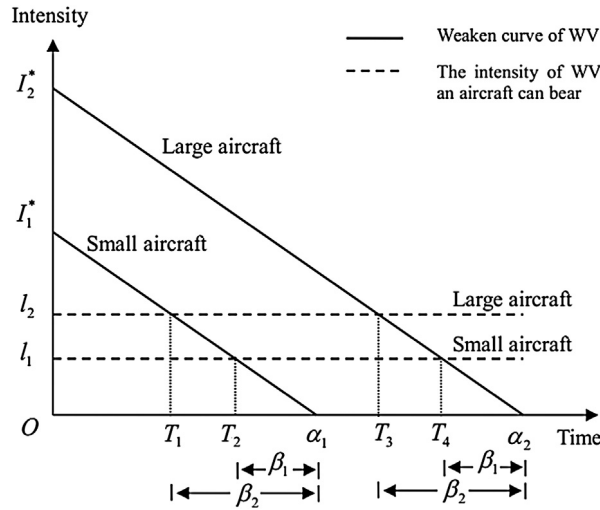


Fig. 2. The WV weakening curve and the intensity of WV an aircraft can bear (involving two types of aircraft).

time x_{π_i} of each aircraft π_i can be calculated by

$$x_{\pi_i} = \begin{cases} \max\{e_{\pi_i}, \text{current time}\}, & \text{if } i = 1; \\ \max\{e_{\pi_i}, x_{\pi_{i-1}} + d_{TP(\pi_{i-1})TP(\pi_i)}\}, & \text{if } 1 < i \leq n. \end{cases} \quad (1)$$

The objective function in this paper is the makespan (W), which is the landing time of the last aircraft in the scheduled sequence π (denoted by π_{end}), that is,

$$(\text{MIP}^1) \min W_\pi = x_{\pi_{\text{end}}}, \quad \forall \pi_i \in \Theta. \quad (2)$$

3. Wave-vortex modeling (WVM) method

The WVM method is originally proposed by modeling the wake-vortex. Firstly, it generates a new (rank 2) MST matrix (denoted by MST'), and the ASP concerning this new MST' is proved to be linearly solvable. Then this new MST' is limitlessly approximating the actual MST. If MST and MST' get close enough (some elements are the same), the solution of ASP concerning the new MST' is just the (hypo-)optimal solution of the ASP concerning the actual MST, i.e. the traditional ASP concerning MST is transferred to an easier ASP concerning MST' . Following are the MST' generating process and the properties of ASP concerning MST' .

3.1. Construction of WVM method

A. The wave-vortex and its weakening curve

The wake-vortex (WV) is a typical air fluctuation, which is generated by the leading aircraft in the air. The WV, if has not been weakened enough, may result in the instability of the following aircraft (to shake or to lift) (Balakrishnan and Chandran [24]). To avoid this, the MST (see in Table 1) is strictly kept between the successive aircraft.

Different types of aircraft always generate and tolerate the WVs of different intensity. Generally, the larger aircraft generates bigger WV and tolerates more turbulent air; while smaller aircraft generates smaller WV and tolerates less turbulence (see in Fig. 2). That is why in Table 1 a smaller aircraft following a larger aircraft requires a much longer separation than the other way around.

The WV is weakening all the time since it has been generated. The weakening process of WV is described by the weakening curve in Fig. 2. $I_1^* \alpha_1$ and $I_2^* \alpha_2$ are two weakening curves for two WVs generated by two types of aircraft (small and large), respectively.

In the WV weakening curve, the intensity of WVs is decreasing as time passes. For example, the small aircraft originally generates the WV of intensity I_1^* ; then the WV weakens; at last, it disappears at the time of α_1 (see the curve $I_1^* \alpha_1$ in Fig. 2). The life span of this WV is α_1 .

B. Life span measures the intensity of wake-vortex

To simplify the discussion, we suppose that all WVs weaken with the same speed (all weakening curves have the same slope k). Then the life spans α of all WVs correspond to their original intensity I^* .

$$I^* = -k \cdot \alpha, \quad \alpha, I^* > 0. \quad (3)$$

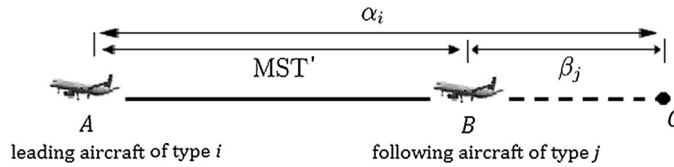


Fig. 3. Life span α , ahead time β and MST' .

For example, in Fig. 2, we have $I_1^* = -k\alpha_1$ and $I_2^* = -k\alpha_2$, where I_1^* and I_2^* are the maximum intensity of the two WVs; α_1 and α_2 are their life span.

Since k is constant, Eq. (3) means that the life span α is equivalent to I^* for the measure of the original (max) intensity of a WV. For example, α_1 and α_2 in Fig. 2 can be used as measures of the original intensity for the two WVs generated by small and large aircraft, respectively. $\alpha_2 > \alpha_1$ because the WV generated by a large aircraft is bigger than that by a small aircraft.

C. Ahead time measures aircraft ability to bear wake-vortex

In addition to generate WV, aircraft have the ability to bear some disturbance of the WV generated by its former aircraft because of its inertia, technical design, and so on. Suppose an aircraft i can bear the WV of the intensity level l_i . In Fig. 2, small aircraft can bear the WV of intensity not bigger than level l_1 , and large aircraft can bear not bigger than level l_2 .

Since the parameter k is constant, the intensity level corresponds to the time β (we call it ahead time). In Fig. 2, $l_1 = -k\beta_1$, $l_2 = -k\beta_2$, $\beta_2 > \beta_1$, because larger aircraft commonly tolerates more turbulent air than small aircraft.

Actually, ahead time means that the aircraft can take off/land ahead of β_j seconds before the disappearance of the WV. That is to say, if an aircraft (of type j) waits for the disappearance of the WV generated by its leading aircraft (of type i), it should take off/land α_i seconds after the leading aircraft; however, it can bear some disturbance of the WV, so it can take off/land β_j seconds ahead, that is, $(\alpha_i - \beta_j)$ seconds after the leading aircraft (see in Fig. 3).

D. Life span minus ahead time is MST'

Consider an aircraft of type i followed by an aircraft of type j . As discussed before, the life span α_i concerns the WV generated by the leading aircraft, and the ahead time β_j concerns the ability of the following aircraft to bear WV. It is easy to confirm that $(\alpha_i - \beta_j)$ is highly relating to the minimum separation time (MST) between these two aircraft. We use MST'_{ij} to denote $(\alpha_i - \beta_j)$ (see in Fig. 3). However, $(\alpha_i - \beta_j)$ is not the actual MST, because many factors, such as aircraft design, airflow, and other disturbance, affect the real MST in need.

E. Three main types of ASP generated by WVM

We use ΔMST to denote the differences between MST and MST' , i.e. $\Delta MST = MST - MST'$. All elements in ΔMST are restricted nonnegative. Then, three main types of aircraft scheduling problem (ASP) arise, which are ASP concerning MST (ASP-MST), ASP concerning MST' (ASP- MST'), and ASP concerning ΔMST (ASP- ΔMST).

ASP-MST means that in the ASP the separation between the successive aircraft is determined by the separation time table MST; ASP- MST' means that the separation is determined by MST' ; and ASP- ΔMST means by ΔMST .

In the following, d_{rs} , d'_{rs} and Δd_{rs} are used to denote the element in the matrices of MST, MST' and ΔMST when an aircraft with type r is followed by another aircraft with type s . We have $\Delta d_{rs} = d_{rs} - d'_{rs}$.

3.2. Important property of ASP- MST'

Although the MST' generated above is not the actual MST, it is a rank 2 matrix and has two main properties to make scheduling easier. First, ASP- MST' is the lower bound of the ASP-MST problem. Second, ASP- MST' is a sequence-independent scheduling problem (while ASP-MST is sequence-dependent), so it is easy to find the optimal solution of ASP- MST' . When optimizing the original ASP concerning MST, we can firstly find its lower bound via ASP- MST' , and then approximating this lower bound to get a good solution.

The ready time constraint is ignored in the discussion in this subsection, and it will be added in the optimization of ASP-MST in Section 4 when designing the AC algorithm. The above two properties are summarized in Theorems 1 and 2 (see the proofs of these two theorems in Ma et al. [27]). Here W_π , W'_π and ΔW_π are the makespan of sequence π in ASP-MST, ASP- MST' and ASP- ΔMST , respectively. Λ is an aircraft type set. When considering Table 1, we have $\Lambda = \{1(\text{small}), 2(\text{large}), 3(\text{heavy})\}$.

Theorem 1. The makespan of each sequence π in ASP- MST' is the lower bound of π in ASP-MST, that is, $W'_\pi \leq W_\pi$.

Theorem 2. The optimal solution of minimizing makespan for aircraft scheduling problem concerning MST' only depends on the first and the last aircraft in the final sequence, if the ready time constraint is ignored and α_i and β_i are given constant, $\forall i \in \Lambda$.

Theorems 1 and 2 simplify the optimization process of ASP- MST' . Only the first and last aircraft in the final sequence should be considered when minimizing the makespan without the ready time constraint. Actually, ASP- MST' is also easy to solve even when the ready time constraint is added, since ASP- MST' is a sequence-independent problem. Theorem 3 presents sufficient condition of the optimal solution of ASP-MST.

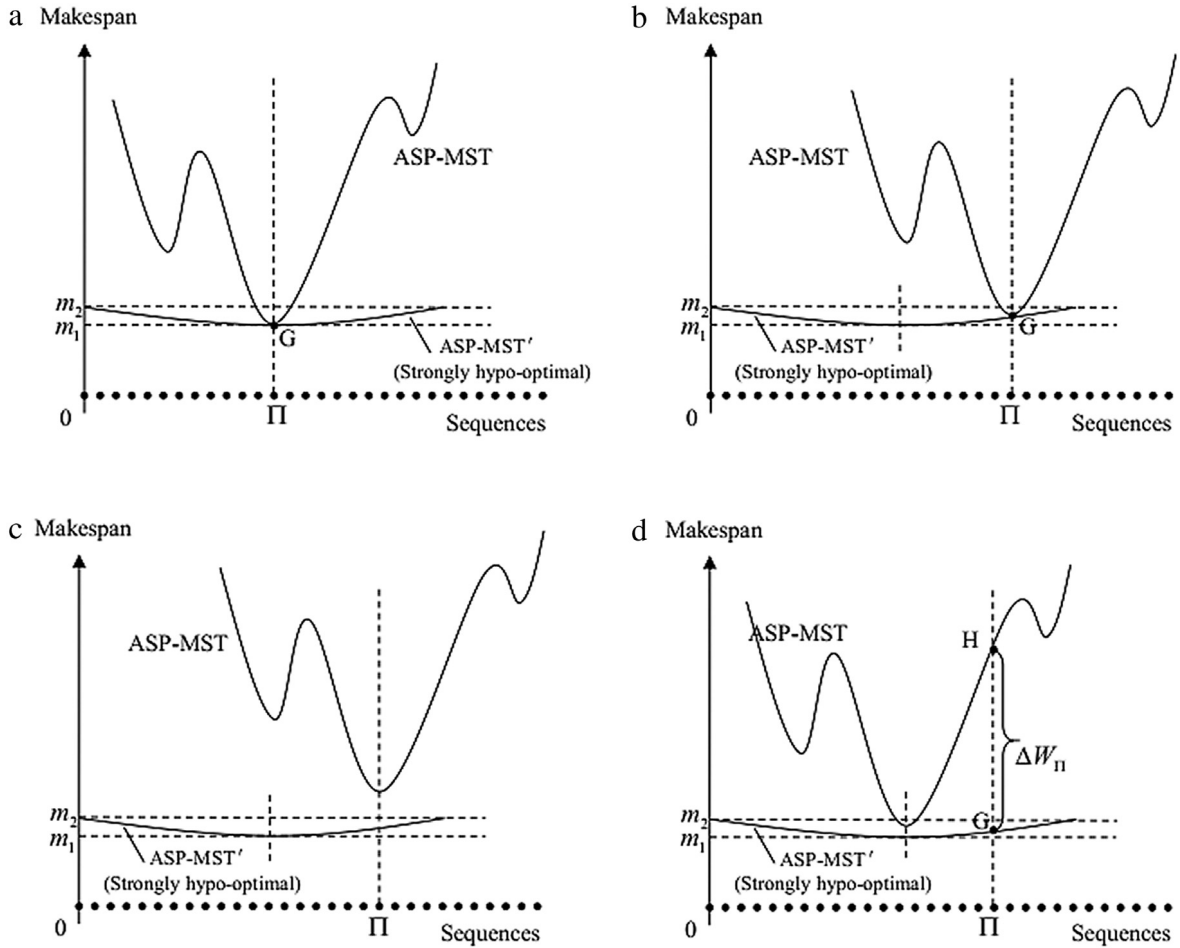


Fig. 4. Makespan of sequences in ASP-MST and ASP-MST'.

Theorem 3. The sequence Π is optimal (with minimized makespan) in ASP-MST model without ready time constraint if the following three requirements are satisfied.

- (1) Sequence Π is the optimal sequence in ASP-MST' model;
- (2) Sequence Π has 0 makespan in ASP- Δ MST model, i.e. $\Delta W_{\Pi} = 0$;
- (3) All elements in Δ MST are nonnegative, i.e. all elements in MST' are not bigger than the corresponding elements in MST.

Proof. For any sequence π , W_{π} denotes the makespan of π in ASP-MST, W'_{π} denotes the makespan in ASP-MST', and ΔW_{π} denotes the makespan of π in ASP- Δ MST.

Requirement (3) means $W'_{\pi} \leq W_{\pi}$ for any given sequence π (Theorem 1).

Requirement (1) means that W'_{Π} is the lower bound makespan of W'_{π} . So for any sequence π , $W'_{\Pi} \leq W'_{\pi} \leq W_{\pi}$.

Requirement (2) means that $W_{\Pi} = W'_{\Pi}$, because when ignoring the ready time constraint, we have

$$\begin{aligned}
 W_{\Pi} - W'_{\Pi} &= \sum_{i=1}^{n-1} d_{\text{TP}(\Pi_i), \text{TP}(\Pi_{i+1})} - \sum_{i=1}^{n-1} d'_{\text{TP}(\Pi_i), \text{TP}(\Pi_{i+1})} \\
 &= \sum_{i=1}^{n-1} (d_{\text{TP}(\Pi_i), \text{TP}(\Pi_{i+1})} - d'_{\text{TP}(\Pi_i), \text{TP}(\Pi_{i+1})}) \\
 &= \sum_{i=1}^{n-1} \Delta d_{\text{TP}(\Pi_i), \text{TP}(\Pi_{i+1})} = \Delta W_{\Pi} = 0.
 \end{aligned}$$

In conclusion, for any sequence π , we have $W_{\Pi} \leq W_{\pi}$, i.e. sequence Π is optimal for ASP-MST. $\square$

Actually, the relationship between ASP-MST and ASP-MST' can be illustrated in Fig. 4(a). Requirement (3) means that ASP-MST' curve is always below ASP-MST curve. Requirement (2) means that the ASP-MST curve is tangent to the ASP-MST'

curve at the point denoting sequence Π . Requirement (1) means that the ASP-MST' curve reaches the bottom at the point denoting sequence Π . So at the point Π , ASP-MST is minimized.

If a sequence Π only satisfies requirements (2) and (3), it is strongly hypo-optimal. Here, “strongly hypo-optimal” means that the sequence's optimality only relates the first and the last aircraft. So a sequence in ASP-MST' without ready time constraint is at least strongly hypo-optimal (Theorem 2). Strongly hypo-optimal solution is illustrated in Fig. 4(b), where $|m_1 - m_2|$ is very small.

4. Ant colony algorithm based on WVM method

As shown before, WVM method can construct rank 2 matrices (MST'). In this section, we generate a MST' matrix to approximate the actual MST matrix, thus the solution of MST is approximating to its lower bound achieved by ASP-MST' (see in Theorem 1). Then, we can use the ASP-MST' for scheduling which is sequence-independent and thus easier to optimize, but a significant loss of precision may occur between MST and MST'. To overcome it, we use the AC algorithm to supplement the precision via considering Δ MST (the differences between MST and MST') and the ready time constraint. Numerical results in Section 5 validate the efficiency of this new AC algorithm, which is called WVM-AC.

Following, in Section 4.1, the MST matrix is approximated by MST', then the WVM-AC algorithm is constructed in Sections 4.2–4.4.

4.1. Linear programming to determining the MST' matrix

Following, we use a linear programming to determine MST', which is shown in LP^1 (Eqs. (4)–(7)). Here, d_{rs} , d'_{rs} and Δd_{rs} are used to denote the element in the matrices of MST, MST' and Δ MST when an aircraft with type r is followed by another aircraft with type s .

The objective (Eq. (4)) is to minimized the total gap between MST and MST' since Δ MST measures their elements' gap (Eq. (5)).

Eq. (6) calculates out the MST', where each element is decomposed into two parts to model the ability of the leading aircraft to generate the WV (denoted by α_r) and the following aircraft to bear the disturbance (denoted by β_s).

In Eq. (7), the main variables (Δd_{rs} , α_r and β_s) are restricted nonnegative. $\Delta d_{rs} \geq 0$ insures that each element in MST' is not bigger than the corresponding element in MST, which also insures Theorem 1 and requirement (3) in Theorem 3.

$$(LP^1) \min \sum_{r=1}^m \sum_{s=1}^n \Delta d_{rs}; \quad (4)$$

subject to

$$d_{rs} - d'_{rs} = \Delta d_{rs}; \quad \forall r, s \in \Lambda \quad (5)$$

$$\alpha_r - \beta_s = d'_{rs}; \quad \forall r, s \in \Lambda \quad (6)$$

$$\Delta d_{rs} \geq 0, \alpha_r \geq 0, \beta_s \geq 0; \quad \forall r, s \in \Lambda. \quad (7)$$

4.2. WVM-AC algorithm construction

Following, we develop an efficient AC algorithm based on WVM method for ASP. AC algorithm is a famous heuristic algorithm via modeling the ants' behavior of searching for food. It shows great advantages in solving the Traveling Salesman Problem (TSP), which aims to find the shortest tour for a traveler visiting out every node once and only once in a graph, starting and finishing at the same origin node (Dorigo [35], Dorigo and Gambardella [36,37]). The ASP can be transferred to an open TSP (the tour does not return to the origin) with ready time, where the runway is regarded as traveler and aircraft as nodes (Randall [14], Bennell et al. [12], Zhan et al. [17], Bencheikh et al. [16]).

The objective is to minimize makespan in Eq. (2). Here, the ASP is transferred to an open TSP, where the ants do not return back to the origin nodes (aircraft). In the algorithm, we wish the ants to visit out all nodes as soon as possible, which is equivalent to the runway landing as many aircraft as possible. Following is the techniques for AC.

(1) Aircraft sequences classification

In the scheduling process, there is no need for the aircraft with the same aircraft type to overtake each other. So, we put together all the aircraft with the same type according to their order in the FCFS sequence (F) to construct subsequences (F^r). And each time we only choose the first ant-unvisited aircraft in F^r to land to avoid the overtaking of the aircraft with the same type. Since there are p aircraft types, we can get p subsequences, which are $F^1, F^2, \dots, F^p$. Here, $TP(F_i^r) = r$, $r \in \Lambda$, Λ is the aircraft type set, and F_i^r is the i th aircraft in F^r .

Take for example a FCFS sequence of types 1, 3, 2, 2, 1, 2, 1, 3, $\Lambda = \{1, 2, 3\}$. Then there are 3 subsequences. F^1 is constructed by the 1st, 5th and 7th aircraft in FCFS according to their order; F^2 is by the 3rd, 4th and 6th aircraft; and F^3 is by the 2nd and 8th aircraft.

(2) Ants Initialization. The initial pheromone in each arc is set the same, that is, $\tau_0(i, j) = 1, \forall i \neq j$. Here i and j denote the nodes (aircraft), and arc is imagined between nodes (aircraft). For each ant s , we use $L^s = \{L_1^s, L_2^s, \dots, L_p^s\}$ to denote the possible aircraft for the ant's next visiting. At the beginning, $L_r^s = F_1^r$, and each ant s randomly chooses an aircraft in $\{F_1^1, F_1^2, \dots, F_1^p\}$ to visit. If F_1^r is visited by ant s , then we set $L_r^s := L_r^s + 1$, that is, $L_r^s = F_2^r$.

(3) State Transition Rule. For each ant s , after visiting aircraft i , it chooses the next allowable aircraft j to visit according to the probability of $Pb(i, j)$.

$$Pb(i, j) = \begin{cases} \frac{[\tau(i, j)]^a [\eta(i, j)]^b}{\sum_{u \in L^s} [\tau(i, u)]^a [\eta(i, u)]^b}, & \text{if } j \in L^s \\ 0, & \text{otherwise} \end{cases} \quad (8)$$

where L^s is a set of allowable aircraft followed by aircraft i . $\tau(i, j)$ is the pheromone in arc (i, j) , and $\eta(i, j)$ denotes the heuristic information in arc (i, j) . The parameters a and b determine the relative importance of τ versus η . We set the heuristic information as

$$\eta(i, j) = \frac{1}{\Delta d_{TP(i)TP(j)} + \max(0, e_j - x_i) + 1}, \quad (9)$$

because there are two main factors that influence the final makespan, which are the nonnegative number in ΔMST and the ready time constraint (differences between the ready time of aircraft j and the landing time of aircraft i). To avoid the infeasible case of 0, the denominator is set as $(\Delta d_{TP(i)TP(j)} + \max(0, e_j - x_i) + 1)$.

4.3. Pheromone updating rules

After all ants complete the tour, the pheromone updating operations are carried out on each arc (i, j) as Eq. (10).

$$\tau_{ij}^{t+1} = (1 - \gamma) \tau_{ij}^t + \Delta \tau_{ij} \quad (10)$$

where τ_{ij}^t is the original pheromone in arc (i, j) and τ_{ij}^{t+1} denotes the current pheromone after the pheromone updating. $(1 - \gamma) \tau_{ij}^t$ models the evaporation of the pheromone, where $(1 - \gamma)$ is the evaporation ratio. $\Delta \tau_{ij}$ is the pheromone lately released by ants, and $\Delta \tau_{ij} = \sum_s \Delta \tau_{ij}^s$. For an ant s , $\Delta \tau_{ij}^s$ is defined as

$$\Delta \tau_{ij}^s = \begin{cases} \frac{Q}{x_{\pi_{\text{end}}}^s}, & \text{if arc } (i, j) \text{ is visited by ant } s \\ 0, & \text{otherwise} \end{cases} \quad (11)$$

where $x_{\pi_{\text{end}}}^s$ is the makespan of the tour by ant s , and Q is a given constant.

4.4. Complete AC algorithm

Step 1. Initialization. m ants touring between aircraft is treated the same as the touring in graph, where the aircraft are regarded as cities, and the MST and ready time constraint compose the arc lengths between cities. The aircraft sequence is divided into p subsequences according to Section 4.2(1). For each ant s , $L^s = \{L_1^s, L_2^s, \dots, L_p^s\}$ denotes the possible aircraft for its next visiting. At the beginning, each ant s randomly chooses an aircraft in L^s and visits it. If L_i^s is chosen, we set $L_i^s := L_i^s + 1$.

Step 2. Each ant s randomly visits the next aircraft in L^s according to Eq. (8). After ant s finishes visiting aircraft L_i^s , we set $L_i^s := L_i^s + 1$. If all aircraft in subsequence F^i have been visited by ant s , we delete L_i^s from set L^s .

Step 3. For ant s , if all aircraft in FCFS have been visited ($L^s = \emptyset$), go to Step 4; otherwise ($L^s \neq \emptyset$), go to Step 2.

Step 4. Calculate out the tour length $x_{\pi_{\text{end}}}^s$ for each ant s by Eq. (1), then find the shortest tour $\min_s x_{\pi_{\text{end}}}^s$. Compare this tour with the historical shortest tour, and reserve the shorter one.

Step 5. Update the pheromone τ_{ij} for all arcs by Eqs. (10) and (11).

Step 6. Termination check. If the termination criteria, such as the limit time, are met, then stop the algorithm and return the shortest tour as the final result. Otherwise, go to Step 1.

5. Numerical study

This section presents the numerical result of the AC-WVM algorithm for ASP. The CPLEX, general AC (without WVM method) algorithm and approximation algorithm in Ma et al. [27] are used as a valid comparison of this new algorithm. Matlab 7.11.0 and CPLEX Optimization Studio 12.5 are adopted here for programming. The implementations are carried out in the personal computer with Intel(R) Core(TM) i3-2310M CPU @ 2.10 GHz 2.10 GHz, RAM 4.00 GB and Windows 7 OS.

Table 2

MST (in sec.) between operations on the same runway for four aircraft types (Bianco et al. [6]).

		Following aircraft types			
		1	2	3	4
Leading aircraft types	1	90	80	70	72
	2	110	80	70	72
	3	130	100	70	72
	4	228	200	181	96

1—Mc Donnell Douglas DC9, 2—Boeing 727, 3—Boeing 707, 4—Boeing 747.

Table 3

Approximation Algorithm (AA), AC and WVM-AC to minimize makespan with the time limit of 15 s.

Data			FCFS	Optimal		AC		AA		WVM-AC	
No.	Type	RT	ALT	No.	ALT	No.	ALT	No.	ALT	No.	ALT
1	1	0	0	1	0	1	0	1	0	1	0
2	1	79	96	3	144	2	96	2	96	2	96
3	1	144	192	2	240	3	192	3	192	3	192
4	2	204	392	5	336	4	392	5	288	5	288
5	1	264	464	4	536	5	464	6	384	6	384
6	1	320	560	6	608	6	560	4	584	4	584
7	2	528	760	7	808	7	760	8	656	7	664
8	1	635	832	9	888	8	832	11	790	8	736
9	2	730	1032	10	968	9	1032	12	920	11	832
10	2	766	1112	8	1040	10	1112	13	1101	12	928
11	1	790	1184	11	1136	11	1184	7	1201	9	1128
12	1	920	1280	12	1232	12	1280	9	1281	13	1198
13	3	1046	1461	13	1413	13	1461	10	1361	10	1298
14	4	1106	1591	15	1513	14	1591	15	1441	15	1378
15	2	1136	1671	14	1623	15	1671	16	1521	16	1458
16	2	1166	1751	17	1703	16	1751	17	1601	17	1538
17	2	1233	1831	16	1783	17	1831	18	1673	14	1648
18	1	1642	1903	20	1853	18	1903	19	1769	18	1720
19	1	1715	1999	18	1925	19	1999	20	1950	19	1816
20	3	1770	2180	19	2021	20	2180	14	2080	20	1997
21	1	2074	2252	21	2117	21	2252	21	2152	21	2074
22	1	2168	2348	23	2345	22	2348	22	2248	22	2170
23	4	2259	2576	22	2417	23	2576	24	2448	23	2398
24	2	2427	2656	25	2513	24	2656	25	2520	24	2478
25	1	2481	2728	24	2713	25	2728	26	2720	25	2550
26	2	2679	2928	26	2793	26	2928	23	2830	26	2750
27	3	2883	2998	27	2883	27	2998	27	2900	27	2883
28	2	2982	3098	28	2983	28	3098	28	3000	28	2983
29	1	3046	3170	29	3055	29	3170	29	3072	29	3055
30	1	3091	3266	30	3151	30	3266	30	3168	30	3151
Makespan			3266	3151		3266		3168		3151	
CPU time (s)						15		0.1		15	

Note: Ready Time (RT), Aircraft Landing Time (ALT).

5.1. Test of aircraft sequence in Hu and Chen [38]

A. Test case

Following, the data in Hu and Chen [38] (see also in Zhan et al. [17]) is used to test the efficiency of the WVM-AC algorithm. The parameters are set as $m = 50$; $a = 1$; $b = 4$; $\gamma = 0.1$; $Q = 100$, and the MST is in Table 2. The termination criterion is set as the time-limitation of 15 s, because the air-traffic controller should make quick decision when scheduling the aircraft and the WVM-AC algorithm can also give a good solution within 15 s.

We compare the WVM-AC with the general AC algorithm and Approximation Algorithm (AA) in Ma et al. [27]. For general AC, the parameters are also set as $m = 50$; $a = 1$; $b = 4$; $\gamma = 0.1$; $Q = 100$. The remaining parameters setting for AC and AA see in Ma et al. [27]. Table 3 illustrates the numerical results. The first 3 column in Table 3 is the data from Hu and Chen [38], where RT means the ready time of each aircraft. ALT denotes the aircraft landing time for each plane. Three main conclusions are summarized from Table 3.

First, the WVM-AC algorithm finds the optimal solution within 15 s, which is better than the results by general AC and approximation algorithm (AA). 15 s is often acceptable for the air-traffic controller to make a quick decision when scheduling the aircraft.

Second, the approximation algorithm (AA) gives a solution with only 0.1 s, but it is not optimal. The general AC algorithm returns the worst solution among these three methods.

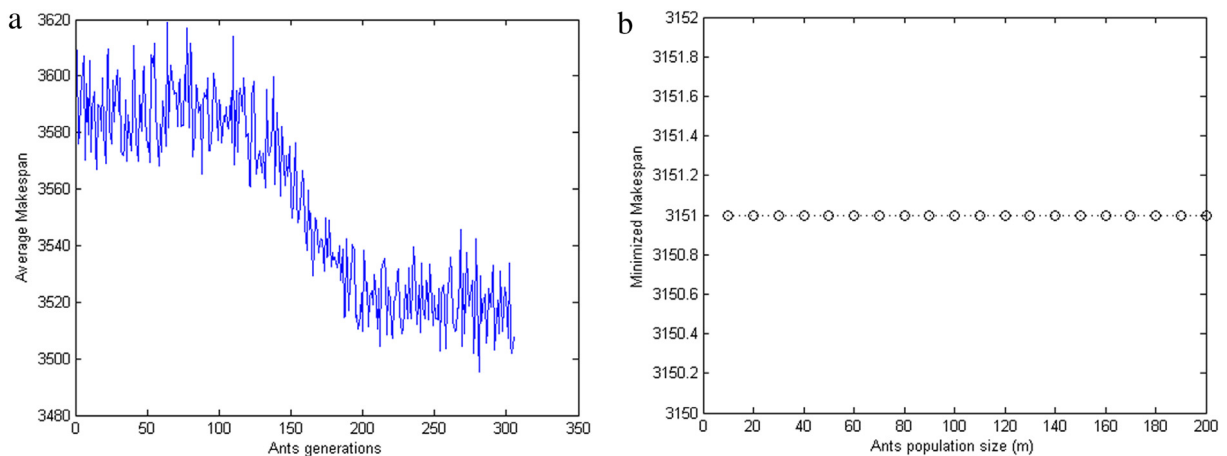


Fig. 5. (a) Relationship between ants generations and the average makespan in each generation; (b) Relationship between ants population size and the minimized makespan by AC in 15 s.

Third, when comparing the FCFS sequence to the final sequence by WVM-AC, we can observe that the aircraft in the final sequence does not shift far away from its position in the FCFS. That is to say, the solution by WVM-AC basically satisfies the requirement of equality.

B. Analysis of the WVM-AC parameters

Following, we analyze the parameters in the WVM-AC algorithm. The parameters include the ants population size (m), ants generations (g) and other parameters of a , b , γ , and Q in Eqs. (8)–(11).

(1) Analysis of the ant-generations

Here, we analyze the influence of ant-generations to the WVM-AC's performance (average makespan). The number of ant-generations has a positive linear correlation with the terminal criteria (runtime-limitation). Here, we also set $m = 50$, $a = 1$, $b = 4$, $\gamma = 0.1$, $Q = 100$. We present in Fig. 5(a) the results by WVM-AC when the ant-generations are set as from 1 to 300 (with the runtime less than 15 s). We can observe that the average makespan in each generation is declining as the number of generations increases. When there are more than 200 generations, the average makespan becomes stable (about 3520). It indicates the convergence of the WVM-AC algorithm for ASP. In this test, 200 generations' iteration costs about 10 s, that is to say, setting the termination criterion to be 15 s in Section 5.1-A is efficient to achieve a good solution.

(2) Analysis of the population size

Here, we analyze the influence of the population size (m) to the WVM-AC's performance (minimized makespan). We set $a = 1$, $b = 4$, $\gamma = 0.1$, $Q = 100$. And the termination criterion is 15 s. We present in Fig. 5(b) the results by WVM-AC when the population size (m) is set as $\{10, 20, 30, 40, \dots, 200\}$. We can observe that in each case, the WVM-AC algorithm finds the optimal solution within 15 s. That is to say, the influence of the population size to the solution is small when the number of aircraft is 30. It is also a verification of the robustness of the WVM-AC algorithm.

(3) Analysis of parameter a

Here, we analyze the influence of the parameter a to the WVM-AC's performance (minimized makespan). We set $m = 30$, $b = 4$, $\gamma = 0.1$, $Q = 100$. And the termination criterion is 15 s. We present in Fig. 6(a) the results by WVM-AC when the parameter (a) is set as $\{0.2, 0.4, 0.6, 0.8, \dots, 4\}$. We can observe that, in each case, the WVM-AC algorithm finds the optimal solution within 15 s. That is to say, the influence of the parameter (a) is small when slightly changing the parameter a . It is also a verification of the robustness of the WVM-AC algorithm.

(4) Analysis of parameter b

Here, we analyze the influence of the parameter b to the WVM-AC's performance (minimized makespan). We set $m = 30$, $a = 1$, $\gamma = 0.1$, $Q = 100$. And the termination criterion is 15 s. We present in Fig. 6(b) the results by WVM-AC when the parameter (b) is set as $\{0.2, 0.4, 0.6, 0.8, \dots, 6\}$. We can observe that, in the case of $b \geq 0.6$, the WVM-AC algorithm finds the optimal solution within 15 s. That is to say, the parameter of (b) should not be set too small when the number of aircraft is 30.

(5) Analysis of parameter γ

Here, we analyze the influence of the parameter γ to the WVM-AC's performance (minimized makespan). We set $m = 30$, $a = 1$, $b = 4$, $Q = 100$. And the termination criterion is 15 s. We present in Fig. 7(a) the results by WVM-AC when the parameter (γ) is set as $\{0.04, 0.08, 0.12, 0.16, \dots, 0.8\}$, since γ should be constrained in set $(0, 1)$. We can observe that, in each case, the WVM-AC algorithm finds the optimal solution within 15 s. That is to say, the influence of the parameter (γ) to the solution is small when slightly changing the parameter γ . It is also a verification of the robustness of the WVM-AC algorithm.

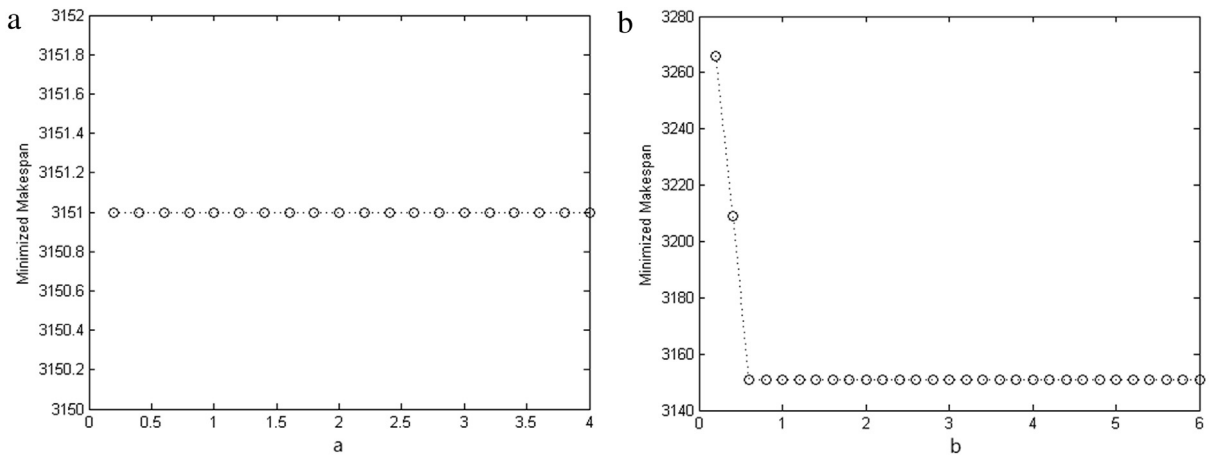


Fig. 6. (a) Relationship between parameter a and the minimized makespan; (b) Relationship between parameter b and the minimized makespan by AC in 15 s.

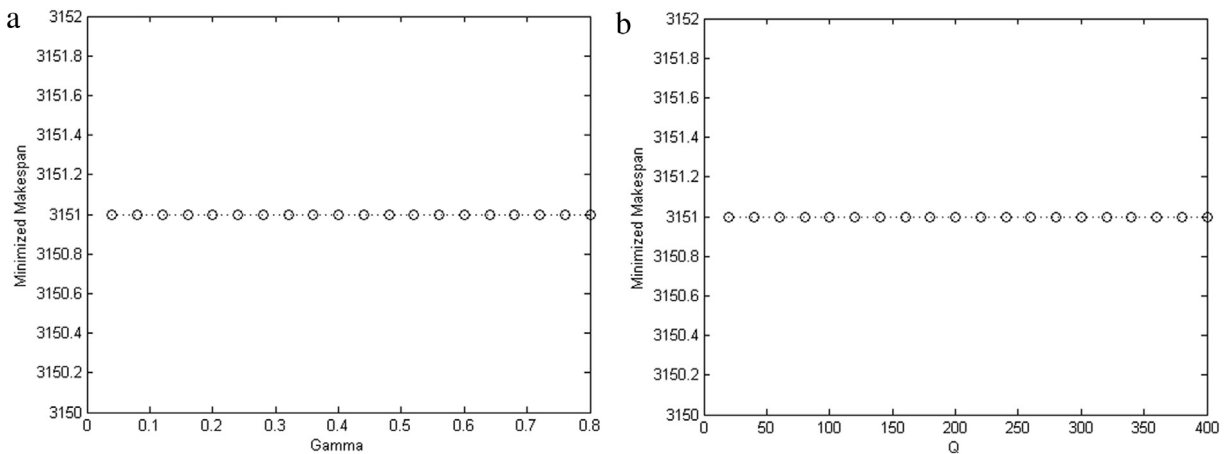


Fig. 7. (a) Relationship between parameter γ and the minimized makespan; (b) Relationship between parameter Q and the minimized makespan by AC in 15 s.

(6) Analysis of parameter Q

Here, we analyze the influence of the parameter Q to the WVM-AC's performance (minimized makespan). We set $m = 30$, $a = 1$, $b = 4$, $\gamma = 0.1$. And the termination criterion is 15 s. We present in Fig. 7(b) the results by WVM-AC when the parameter (Q) is set as $\{20, 40, 60, 80, \dots, 400\}$. We can observe that, in each case, the WVM-AC algorithm finds the optimal solution within 15 s. That is to say, the influence of the parameter (Q) to the solution is small. It is also a verification of the robustness of the WVM-AC algorithm.

5.2. Test of aircraft sequence by Poisson arrival process

Following, we use the WVM-AC to optimize the aircraft sequence generated by Poisson arrival process, which has been validated by Willemain et al. [31].

A. Generate a general MST in numerical study

The MST table is important in WVM method. Tables 1 and 2 present two different MST tables by two classical classifications of aircraft types. And in the Next Generation Air Transportation System, there are more than 3 categories of aircraft. We believe that our algorithm applies almost all possible MST matrices. So, the MST matrices are randomly generated in this test. We use $D_{p \times p}$ to denote a general MST.

$$D_{p \times p} = \begin{bmatrix} d_{11} & d_{12} & \cdots & d_{1p} \\ d_{21} & d_{22} & \cdots & d_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ d_{p1} & d_{p2} & \cdots & d_{pp} \end{bmatrix} \quad (12)$$

where p is the number of aircraft types. There are two general properties for $D_{p \times p}$.

Table 4

Approximation Algorithm (AA), CPLEX, AC and WVM-AC to minimize makespan with the time limit of 15 s.

Aircraft types	Number of aircraft	Time (s)	Runway enhance (%)			
			AA ^a	CPLEX	AC	WVM-AC
$p = 3$	20	15	8.93	<u>13.48</u>	5.19	10.91
	50	15	10.72	<u>10.77</u>	4.60	<u>12.60</u>
	80	15	6.44	6.22	3.08	<u>11.00</u>
	110	15	8.38	5.12	4.22	<u>13.37</u>
	140	15	12.33	2.25	3.13	<u>14.52</u>
	170	15	8.38	1.57	3.93	<u>14.14</u>
	200	15	9.00	0.09	3.28	<u>15.17</u>
	230	15	10.52	0	3.27	<u>14.20</u>
	260	15	11.94	0	4.52	<u>15.68</u>
$p = 9$	20	15	2.72	<u>9.81</u>	2.49	9.04
	50	15	5.42	9.00	5.14	<u>14.22</u>
	80	15	7.20	4.81	2.91	<u>12.05</u>
	110	15	5.31	3.30	2.01	<u>11.21</u>
	140	15	2.93	0.85	1.51	<u>11.31</u>
	170	15	5.03	0.37	1.75	<u>11.09</u>
	200	15	5.22	0.56	2.79	<u>12.56</u>
	230	15	7.25	0.15	2.29	<u>13.18</u>
	260	15	6.84	0	1.55	<u>12.14</u>

Underlined figures mark the maximum runway enhancement.

^a Here, AA algorithm costs less than 0.1 s to optimize each sequence.

(1) Larger aircraft before the same aircraft needs more (at least equal) separation, i.e. $d_{ij} \geq d_{i-1,j}$, $i = 2, 3, \dots, p$; $j = 1, 2, \dots, p$.

(2) Larger aircraft after the same aircraft needs less (at most equal) separation, i.e. $d_{ij} \geq d_{i,j+1}$, $i = 1, 2, \dots, p$; $j = 1, 2, \dots, (p-1)$.

In the numerical study, the matrix $D_{p \times p}$ is generated by

$$d_{ij} = \begin{cases} k_{ij}d_{i,j+1}, & \text{if } i = 1; j = 1, 2, \dots, p-1 \\ k_{ij}d_{i-1,j}, & \text{if } i = 2, 3, \dots, p; j = p \\ k_{ij} \cdot \max\{d_{i,j+1}, d_{i-1,j}\}, & \text{otherwise} \end{cases} \quad (13)$$

where d_{1p} is a given number, and $k_{ij} \geq 1$ can be randomly set.

B. Numerical results

We test the cases of $p = 3$ and $p = 9$. In each case, 9 groups of aircraft sequences are randomly generated, which are $n = 20, 50, 80, \dots, 260$. Each group contains 5 random FCFS sequences. In each sequence, the number of all types of aircraft is the same. The MST is randomly generated according to Eq. (13). When the aircraft types $p \leq 4$, k_{ij} (in Eq. (13)) is a uniform random number in the interval of $[1, 2]$; when $p > 4$, it is in the interval of $[1, 1 + \frac{1}{(p-3)}]$, which insures d_{p1} not too much bigger than d_{1p} . This setting is very close to the actual MST; for example, Tables 1 and 2 basically satisfy this setting.

For each aircraft, the Estimated Landing Time (ELT) is simulated by a Poisson arrival process, which has been validated by Willemain et al. [31]. The expectation of interval between neighbor ELT is set as $\frac{4}{5}E(D_{p \times p})$, which expresses how crowded an airport is. Here $E(D_{p \times p})$ denotes the mean value of all elements in $D_{p \times p}$. The ready time (earliest landing time) is set as one minute less than the ELT, since more than one minute forward move is often uneconomical (Neuman and Erzberger [21]).

For the WVM-AC and general AC algorithm, the parameters are set as $m = 30$; $a = 1$; $b = 4$; $\gamma = 0.1$; $Q = 100$. For the approximation algorithm, general AC algorithm and CPLEX, the remaining parameters are set the same as in Ma et al. [27] with the latest landing time of $+\infty$ and the terminal condition (limit time) of 15 s (Table 4) and of 120 s (Table 5). That is because the ASP decision should be given quickly in the real world application. We also present the results in 3600 s to show how the solution close to the optimal solution. The numerical results include runtime and runway enhance by each algorithm. The traditional AC involving CPS constraint to insure equity, and the WVM-AC can also basically insure the equity requirement (see in Table 3). The “runway enhance” is computed by $\frac{W_{FCFS} - W_{\pi}}{W_{FCFS}} \times 100\%$ for each sequence. It is a comparison of the makespan between the final scheduled sequence and the FCFS sequence since FCFS is widely used in current runway system.

(1) Time limit of 15 s.

Table 4 presents the results (average runway enhance to 5 random sequences in each group) with the terminal criteria (limit time) of 15 s, since the aircraft scheduling decision should be made quickly in the real world application.

No matter when the number of aircraft types is 3 or 9, the WVM-AC algorithm enhances the runway throughput most except the case of $n = 20$. It validates the efficiency of the algorithm. Actually, classifying the aircraft into more than 9 types is often unnecessary, which will greatly increase the workload of the air-traffic controller. So the results for the condition of $p > 9$ are not discussed here. In the case of $n = 20$, CPLEX often finds the optimal solution since the problem size is not very big.

Table 5

Approximation Algorithm (AA), CPLEX, AC and WVM-AC to minimize makespan with the time limit of 120 s.

Aircraft types	Number of aircraft	Time (s)	Runway enhance (%)			
			AA ^a	CPLEX	AC	WVM-AC
$p = 3$	20	120	6.07	<u>9.57</u>	4.69	8.06
	50	120	8.70	<u>9.84</u>	4.06	<u>12.48</u>
	80	120	7.14	<u>11.22</u>	4.23	<u>14.43</u>
	110	120	9.46	<u>7.36</u>	3.80	<u>16.45</u>
	140	120	13.55	<u>9.15</u>	5.49	<u>18.82</u>
	170	120	8.10	<u>3.44</u>	3.10	<u>10.48</u>
	200	120	8.96	<u>5.54</u>	5.29	<u>13.14</u>
	230	120	8.91	<u>3.77</u>	4.82	<u>15.82</u>
$p = 9$	260	120	10.53	<u>1.85</u>	5.55	<u>15.58</u>
	20	120	2.94	<u>11.70</u>	8.33	10.48
	50	120	2.47	<u>7.89</u>	1.52	<u>9.00</u>
	80	120	3.70	<u>8.36</u>	3.55	<u>11.85</u>
	110	120	3.63	<u>6.56</u>	2.62	<u>11.40</u>
	140	120	4.83	<u>3.79</u>	1.72	<u>10.23</u>
	170	120	4.70	<u>2.01</u>	1.88	<u>11.30</u>
	200	120	6.33	<u>2.50</u>	2.89	<u>12.13</u>
	230	120	2.69	<u>1.71</u>	1.49	<u>10.13</u>
	260	120	3.50	<u>1.31</u>	1.92	<u>9.64</u>

Underlined figures mark the maximum runway enhancement.

^a Here, AA algorithm costs less than 0.1 s to optimize each sequence.**Table 6**

WVM-AC and CPLEX to minimize makespan with time limit of 120 and 3600 s.

p	Number of aircraft	FCFS	WVM-AC (120 s)		CPLEX (3600 s)	
			Time (s)	Runway enhance (%)	Time (s)	Runway enhance (%)
$p = 3$	20	3 247	120	1.69	1.3	<u>2.82</u> ^a
	50	7 445	120	11.11	3600	11.41
	80	13 051	120	<u>15.71</u>	3600	13.83
	110	12 577	120	<u>20.11</u>	3600	14.94
	140	21 127	120	<u>17.46</u>	3600	12.44
	170	25 228	120	<u>11.49</u>	3600	7.68
	200	31 466	120	<u>23.96</u>	3600	17.38
	230	36 294	120	<u>15.90</u>	3600	9.71
$p = 9$	260	48 818	120	<u>12.91</u>	3600	8.68
	20	2 201	120	12.32	653	<u>13.29</u> ^a
	50	5 956	120	<u>5.45</u>	3600	4.93
	80	8 954	120	<u>5.81</u>	3600	5.04
	110	13 399	120	<u>8.01</u>	3600	6.77
	140	16 706	120	<u>8.77</u>	3600	6.76
	170	18 789	120	<u>10.93</u>	3600	7.76
	200	24 078	120	<u>11.56</u>	3600	9.32
	230	28 821	120	<u>9.64</u>	3600	4.82
	260	32 654	120	<u>11.39</u>	3600	7.15

Underlined figures mark the maximum “runway enhance”.

^a Marks the optimal value.

By analyzing the approximation algorithm, AC and CPLEX, we find that CPLEX has better performance when the number of aircraft is small. For a large number of aircraft, it better to use the approximation algorithm for schedule. The efficiency of CPLEX is declining when the number of aircraft (n) becomes larger, since the problem size is increasing.

(2) Time limit of 120 s.

Table 5 presents the results (average runway enhance) with the terminal criteria (limit time) of 120 s, which is a more relaxed time for computation. The main results are similar as that in Table 4.

First, no matter when the number of aircraft types is 3 or 9, the WVM-AC algorithm enhances the runway throughput most except the case of $n = 20$. It validates the efficiency of the algorithm. In the case of $n = 20$, CPLEX often finds the optimal solution since the problem size is not very big.

Second, by analyzing the approximation algorithm, AC and CPLEX, we find that CPLEX has better performance when the number of aircraft is small. For a large number of aircraft, it better to use the approximation algorithm for schedule. The efficiency of CPLEX is declining when n becomes larger, since the problem size is increasing.

(3) Time limit of 3600 s.

Table 6 presents the results of WVM-AC (with the terminal criteria of runtime 120 s) compared to the solution by CPLEX (with the terminal criteria of 3600 s). In this test, one sequence is randomly generated in each group (row) of aircraft number.

We can observe that, no matter when the number of aircraft types is 3 or 9, the WVM-AC algorithm enhances the runway throughput most except the case of $n = 20, 50$. More important, the time cost by WVM-AC is only 120 s. It validates the efficiency of the algorithm. In the case of $n = 20$, CPLEX finds the optimal solution since the problem size is not big. The efficiency of CPLEX is declining when n becomes larger, since the problem size is increasing.

6. Conclusions and future work

To cope with aircraft scheduling problem (ASP), an Ant Colony algorithm based on WVM method is proposed, which is constructed by modeling the wake-vortex using the rank 2 matrix. Numerical results validate that it has better performance than CPLEX, general AC algorithm, and approximation algorithm in Ma et al. [27].

In the future study, we can combine the WVM methods with other heuristic algorithm for optimization to get higher efficiency. We can also try to apply the WVM-AC algorithm in the Next Generation Air Transportation System (Federal Aviation Administration [39]) or under the condition of the wake vortex re-categorization (Lang et al. [40]). In addition, extending the rank 2 matrix approximation method to solve other combinatorial optimization problems is also challenging and interesting.

Acknowledgments

The authors are deeply indebted to the editors, referees, and Dr. Ming Liu for their invaluable suggestions and comments which greatly improved the presentation of this paper.

The work was partly supported by the Hujiang Fund for Philosophical and Social Science (16HJSK-YB08), Hujiang Fund for Ph.D. Scientific Research Startup (BSQD201605) and the National Natural Science Foundation of China (71371141).

References

- [1] Federal Aviation Administration, (2009). <http://aspm.faa.gov/main/opsnet.asp>.
- [2] D. Boehme, Improved airport surface traffic management by planning, in: H. Winter, H.-G. Nsser (Eds.), *Advanced Technologies for Air Traffic Flow Management*, in: Lecture Notes in Control and Information Science, vol. 198, Springer, Berlin, 1994, pp. 191–224.
- [3] K.D. Arkind, Requirements for a novel terminal area capacity enhancement concept in 2022, in: *American Institute of Aeronautics and Astronautics, Guidance, Navigation and Control Conf. Exhibit*, Providence, RI, American Institute of Aeronautics and Astronautics, Reston, VA, 2004.
- [4] H.R. Idris, B. Delcaire, I. Anagnostakis, W.D. Hall, N. Pujet, E. Feron, R.J. Hansman, J.-P. Clarke, A.R. Odoni, Identification of flow constraint and control points in departure operations at airport systems, in: *American Institute of Aeronautics and Astronautics, Guidance, Navigation and Control Conf.*, Boston, AIAA-1998-4291, American Institute of Aeronautics and Astronautics, Reston, VA, 1998.
- [5] J.E. Beasley, M. Krishnamoorthy, Y.M. Sharaiha, D. Abramson, Scheduling aircraft landings-The static case, *Trans. Sci.* 34 (2) (2000) 180–197.
- [6] L. Bianco, P. Dell'Omo, S. Giordani, Scheduling models and algorithms for TMA traffic management, in: L. Bianco, P. Dell'Omo, A.R. Odoni (Eds.), *Modelling and Simulation in Air Traffic Management*, Springer-Verlag, New York, 1997, pp. 139–167.
- [7] M.R. Garey, D.S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*, Freeman, New York, 1979.
- [8] I. Anagnostakis, J.-P. Clarke, D. Boehme, U. Volckers, Runway operations planning and control: Sequencing and scheduling, *J. Aircr.* 38 (6) (2001) 988–996.
- [9] C.R. Brinton, An implicit enumeration algorithm for arrival aircraft scheduling, in: *Proceedings of the IEEE/AIAA 11th Digital Avionics Systems Conference*, Seattle, WA, USA, 1992.
- [10] J. Abela, D. Abramson, M. Krishnamoorthy, A. De Silva, G. Mills, Computing optimal schedules for landing aircraft, in: *Proceedings of 12th National Conference of the Australian Society for Operations Research*, Adelaide, Australia, 1993, pp. 71–90.
- [11] A.T. Ernst, M. Krishnamoorthy, R.H. Storer, Heuristic and exact algorithms for scheduling aircraft landings, *Networks* 34 (1999) 229–241.
- [12] J.A. Bennell, M. Mesgarpour, C.N. Potts, Airport runway scheduling, *Q. J. Oper. Res.* 9 (2011) 115–138.
- [13] A. Faye, Solving the Aircraft Landing Problem with time discretization approach, *European J. Oper. Res.* 242 (3) (2015) 1028–1038.
- [14] M.C. Randall, Scheduling aircraft landings using ant colony optimisation, in: *Proceedings of The IASTED International Conference Artificial Intelligence and Soft Computing*, Banff, Canada, 2002.
- [15] G. Bencheikh, J. Boukachour, A.E.H. Alaoui, F.E. Khoukhi, Hybrid method for aircraft landing scheduling based on a job shop formulation, *Int. J. Comput. Sci. Netw. Secur.* 9 (2009) 78–88.
- [16] G. Bencheikh, J. Boukachour, A.E.H. Alaoui, Improved ant colony algorithm to solve the aircraft landing problem, *Int. J. Comput. Theory Eng.* 3 (2) (2011) 224–233.
- [17] Z.H. Zhan, J. Zhang, Y. Li, O. Liu, S.K. Kwok, W.H. Ip, O. Kaynak, An efficient ant colony system based on receding horizon control for the aircraft arrival sequencing and scheduling problem, *IEEE Trans. Intell. Transp. Syst.* 11 (2) (2010) 399–412.
- [18] R.G. Dear, The dynamic scheduling of aircraft in the near terminal area, MIT Flight Transportation Laboratory Report R76–9, Massachusetts Institute of Technology, Cambridge, 1976.
- [19] H.N. Psaraftis, A dynamic programming approach for sequencing groups of identical jobs, *Oper. Res.* 28 (6) (1980) 1347–1359.
- [20] R.G. Dear, Y.S. Sherif, An algorithm for computer assisted sequencing and scheduling of terminal area operations, *Trans. Res. Part A, Policy Practice* 25 (2–3) (1991) 129–139.
- [21] F. Neuman, H. Erzberger, Analysis of delay reducing and fuel saving sequencing and spacing algorithms for arrival spacing, NASA technical report A-91203; NAS 1.15:103880; NASA-TM-103880. Retrieved August 2008, NASA Technical Reports Server (NTRS), 1991.
- [22] D.A. Trivizas, Optimal scheduling with maximum position shift (MPS) constraints: A runway scheduling application, *J. Navig.* 51 (2) (1998) 250–266.
- [23] R. de Neufville, A.R. Odoni, *Airport Systems: Planning, Design and Management*, McGraw-Hill, New York, 2003.
- [24] F.R. Carr, Robust decision-support tools for airport surface traffic (Ph.D. thesis), Massachusetts Institute of Technology, Cambridge, MA, 2004.
- [25] H. Balakrishnan, B.G. Chandran, Algorithms for scheduling runway operations under constrained position shifting, *Oper. Res.* 58 (6) (2010) 1650–1665.
- [26] A. Lieder, D. Briskorn, R. Stollitz, A Dynamic Programming Approach for the Aircraft Landing Problem with Aircraft Classes. 2013. <https://ub-madoc.bib.uni-mannheim.de/35852/1/SSRN-id2391111.pdf>.
- [27] W. Ma, B. Xu, M. Liu, H. Huang, An efficient approximation algorithm for aircraft arrival sequencing and scheduling problem, *Math. Probl. Eng.* 2014 (2014) 8. <http://dx.doi.org/10.1155/2014/236756>, Article ID 236756, Published 31 December 2014.
- [28] S. Atkins, C. Brinton, Concept description and development plan for the surface management system, *J. Air Traffic Control* 44 (1) (2002).
- [29] D. Boehme, Tactical departure management with the Eurocontrol/DLR DMAN. 6th USA/Europe ATM R&D Seminar, Baltimore, 2005.

- [30] H. Lee, Tradeoff evaluation of scheduling algorithms for terminal-area air traffic control (Master's thesis), Massachusetts Institute of Technology, Cambridge, MA, 2008.
- [31] T.R. Willemain, H. Fan, H. Ma, Statistical analysis of intervals between projected airport arrivals. DSES Technical Report 38–04–510, Rensselaer Polytechnic Institute, Troy, NY, 2004.
- [32] G. De Maere, J.A.D. Atkin, Pruning rules for optimal runway sequencing with airline preferences, *Lect. Notes Manag. Sci.* 7 (2015) 76–82. <http://orlabanalytics.ca/lnms/archive/v7/lnmsv7p76.pdf>.
- [33] X. Ji, X. Cao, W. Du, K. Tang, An evolutionary approach for dynamic single-runway arrival sequencing and scheduling problem, *Soft Comput.* (2016) <http://dx.doi.org/10.1007/s00500-016-2241-8>.
- [34] A. Lieder, R. Stolletz, Scheduling aircraft take-offs and landings on interdependent and heterogeneous runways, *Trans. Res. Part E Logist. Trans. Rev.* 88 (2016) 167–188.
- [35] M. Dorigo, Optimization, Learning and Natural Algorithms (Ph.D. thesis), Dip. Elettronica, Politecnico di Milano, Italy, 1992.
- [36] M. Dorigo, L.M. Gambardella, Ant colony system: A cooperative learning approach to the traveling salesman problem, *IEEE Trans. Evol. Comput.* 1 (1) (1997) 53–66.
- [37] M. Dorigo, L.M. Gambardella, Ant colonies for the traveling salesman problem, *Biosystems* 43 (1997) 73–81.
- [38] X.-B. Hu, W.-H. Chen, Genetic algorithm based on receding horizon control for arrival sequencing and scheduling, *Eng. Appl. Artif. Intell.* 18 (2005) 633–642.
- [39] Federal Aviation Administration (2016). NextGen Performance Snapshots. <https://www.faa.gov/nextgen/snapshots/>.
- [40] S. Lang, J. Tittsworth, W.H. Bryant, P. Wilson, C. Lepadatu, D.P. Delisi, D.Y. Lai, G.C. Greene, Progress on an ICAO Wake Turbulence Re-Categorization Effort. AIAA Atmospheric and Space Environments Conference 2–5 August 2010, Toronto, Ontario Canada, 2010.